

🎵 Beethoven — 音乐多乐器声源分离

"把谱写好的音乐拆回去，听每个乐器原本在说什么。"

Beethoven is about taking apart what Beethoven put together.

Beethoven 是一个音乐多乐器声源分离（Music Source Separation）项目。输入一首混合完成的歌曲，输出各乐器的独立声轨——人声 🎤、贝斯 🎸、鼓 🥁、吉他 🎸、钢琴 🎹、其他。

项目名致敬路德维希·凡·贝多芬——他谱写交响乐，而本项目试图"解谱"：将混合乐曲逆向拆解为各乐器的独立声源。

🌟 功能亮点

功能	说明
🔑 一键分离	上传歌曲 → 分离为 4/6 个独立乐器声轨
🎛️ 更细粒度	6 轨模式：人声/贝斯/鼓/吉他/钢琴/其他
🎵 MIDI 乐谱	构想二自动生成每件乐器的 MIDI 文件
🌐 双语界面	Web 界面支持 CN 中文 / GB English 切换
🧠 自研模型	U-Net / U-Net++ 完全从零训练，658 万参数

🔬 两种技术构想

构想一：波形分解 + 时序约束 + 先验知识（掩码法）

混合波形是各乐器声波在空气中的**线性叠加**。逆向拆解是典型的多解问题（ill-posed inverse problem），因此需要：

- 短时分解**：STFT 将音频切为 ~20-40ms 的时间微元，每帧求解
- 时序连续性约束**：真实乐器演奏在相邻帧不会突变——这是筛选有效解的关键约束
- 先验知识**：深度学习模型从数据中习得"乐器听起来应该是什么样"

实现：频谱掩码法 + U-Net 架构（对应 Demucs 的主流范式）。

构想二：逐乐器"重演" + 片段筛选（生成式）

受 B 站"钢琴还原名场面"启发——不是从混合信号中"滤出"乐器，而是**让模型用该乐器的音色重新演奏**混合音频中属于它的部分：

Demucs 粗分 → 音高检测 → 乐器分配 → 音色重演 → 置信度筛选 → 🎵 MIDI 导出

副产品是可直接编辑的 MIDI 乐谱——不仅分离了音频，还得到了"哪个乐器在什么时候弹了什么音"。

📊 成果

分离质量演进

方法	钢琴 SDR	小提琴 SDR	贝斯 SDR
NMF (纯信号处理基线)	1.8 dB	0.6 dB	0.7 dB
U-Net (本项目)	13.4 dB	17.4 dB	11.0 dB
提升	+11.6 dB	+16.8 dB	+10.3 dB

在真实歌曲上的表现 (教师-学生学习)

仅用 17 段切片、1 首歌训练，自研 U-Net++ 达到 Demucs 老师 66-105% 的水平：

声轨	Demucs (老师)	U-Net++ (学生)	接近程度
人声	RMS 0.075	RMS 0.059	79%
贝斯	RMS 0.043	RMS 0.042	98%
鼓	RMS 0.044	RMS 0.029	66%
其他	RMS 0.054	RMS 0.057	105%

音符检测 (构想二)

歌曲	检测音符总数	MIDI 文件
BEYOND - 光辉岁月	1,743	6 份
ヨルシカ - 花に亡霊	1,345 (钢琴 245 个)	6 份

 快速开始

环境要求

- Python 3.10+ (当前支持 3.14)
- NVIDIA GPU (可选，CPU 也能跑)

安装

```
pip install demucs librosa soundfile matplotlib torch torchaudio gradio midiutil
```

一键分离 (Web 界面)

```
python code/webapp.py
# 浏览器打开 http://127.0.0.1:7865
# 上传歌曲 → 选择 4轨/6轨/构想二 → 开始分离
```

命令行分离 (Demucs 6 轨)

```
python -m demucs -n htdemucs_6s -o samples/separated "你的歌曲.mp3"
```

构想二：生成式分离 + MIDI

```
python code/approach2_fixed.py
# 输出在 samples/separated/approach2_fixed/
```

📁 项目结构

```
beethoven/
├── code/
│   ├── webapp.py                # 🖥️ Web 界面（中英双语，4轨/6轨/构想二）
│   ├── phase3_teacher_student.py # 🎤 U-Net++ 教师-学生训练（真实歌曲）
│   ├── phase2_unet.py           # 🎤 U-Net 训练（合成数据）
│   ├── approach2_fixed.py       # 🎧 构想二：生成式分离 + MIDI
│   ├── phase1_synthesize_data.py # 合成数据生成（钢琴/小提琴/贝斯）
│   ├── phase1_nmf_baseline.py   # NMF 基线分离
│   ├── dataset.py               # PyTorch 数据管线
│   └── ...
├── docs/                        # 📖 完整技术文档（含开发全程解读）
├── _pdf/                        # 📄 PDF 版本（12 份报告）
├── samples/
│   ├── synthetic/              # 合成训练数据
│   ├── unet_best.pth           # 训练好的 U-Net 权重（194万参数）
│   ├── unetpp_best.pth         # 训练好的 U-Net++ 权重（658万参数）
│   └── spectrogram_comparison.png # 分离效果频谱图
├── ORIGINAL_IDEA.txt           # 项目最初的想法记录
└── README.md
```

📖 文档

文档	内容
docs/technical_foundation.md	信号处理与深度学习基础（STFT、U-Net、SDR...）
docs/approach_1.md	构想一：波形分解法详细设计
docs/approach_2.md	构想二：逐乐器重演法详细设计
docs/roadmap.md	分阶段实现路线图
docs/development_summary.md	开发全程解读 + 构想一技术详解
docs/report_phase0~4.md	各阶段实验报告

🔧 技术栈

领域	工具
音频处理	librosa · soundfile · scipy
深度学习	PyTorch · torchaudio
基线参考	demucs (Meta)
Web 界面	gradio

领域	工具
MIDI	<code>midiutil</code>

声明

- 项目中的测试歌曲（光辉岁月、花に亡霊等）仅用于本地实验，**未包含在仓库中**
- 构想二目前使用简易合成器，音色为"MIDI 风格"——真实的 SoundFont 音源库会大幅提升效果

项目始于一个洗澡时的灵光一闪   欢迎提交 *Issue / PR / Star* 